

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Traditional file systems such as EXT2 store metadata — permissions, timestamps, ownership, and block pointers — inside fixed, statically-sized on-disk structures called inodes. While this design is efficient for basic file access, it has no native support for structured search, dynamic/custom attributes, or analytical queries. Finding all files owned by a given user, filtering by modification date, or tagging a file with custom application-level metadata all require either custom-made tooling or limited extended attributes (xattrs). At the same time, modern large-scale storage systems (Amazon S3, HDFS) address exactly this limitation by decoupling metadata from data — metadata is moved into a fast, structured, queryable index, while raw file content continues to live in a separate, high-throughput storage layer. The goal of this project is to use this knowledge for making own decoupled system using relational database for storing the metadata: combining Operating System concepts with Database Management System concepts into a single, cohesive system.

State of the Art / Current solution (1 pt)

The problem of separating metadata from raw data storage is well established in industry, and this project's design follows the same pattern used by existing production systems:

- **EXT2 / EXT3:** Metadata and data tightly bound in inodes; no query language.
- **Amazon S3 :** Object storage with decoupled metadata index; it uses NoSQL key-value pairs.
- **HDFS:** NameNode separated from DataNodes; requires a heavy distributed multi-node cluster. It uses a dedicated master node.
- **Relational DB with BLOBs:** It is an anti-pattern; large binary data degrade database performance and backups.

Project Goals and Milestones (2 pts)

Goals

- Design and implement a metadata layer using a DBMS to store file paths, permissions, timestamps, block/blob pointers.
- Design and implement a data layer that stores raw file content as blocks/blobs, using block-allocation logic from a EXT2 file system.
- Provide a fast, indexed, multi-attribute SQL query interface over file metadata.
- Implement a write-ahead log (WAL) / journaling mechanism to maintain consistency between layers across simulated crashes.

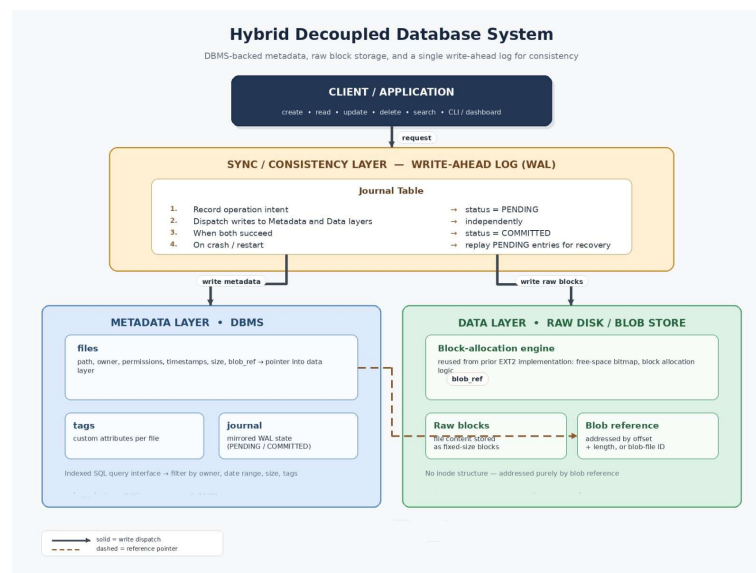
Milestones

- Metadata schema finalized; blob addressing scheme designed
- Data layer functional for blob write, read, and delete operations
- Metadata layer functional with DBMS schema, CRUD, and indexing
- WAL, crash-recovery, and search interface complete
- Final report submitted

Project Approach (3 pts)

- **Step 1:** Study the EXT2 implementation to use block-allocation logic.
- **Step 2:** Implement the data layer where raw content is written to disk blocks or blob files, addressed by blob references.
- **Step 3:** Design and implement the metadata layer schema in MySQL, storing paths, permissions, timestamps.
- **Step 4:** Implement the write-ahead log: operations record status *PENDING* in a journal table before committing to *COMMITTED*.
- **Step 5:** Build a query/search interface demonstrating indexed SQL search over metadata.

System Architecture (High Level Diagram) (2 pts)



Project Outcome / Deliverables (1 pts)

- A working prototype implementing file create, read, update, delete, and search operations across the decoupled architecture.
- A relational metadata schema (files, tags, journal tables) with indexes supporting fast multi-attribute SQL search.
- A functioning write-ahead log demonstrating crash recovery tested via process kill mid-write.
- A demonstration applied to a document management use case showing indexed search.
- Project documentation covering design rationale, schema definitions, and production comparisons (S3, HDFS).

Assumptions

- The system runs on a single machine; no distributed/networked nodes or multi-node replication are implemented.
- Concurrent multi-user access is demonstrated at a basic level only, not under full production concurrency control.
- Full POSIX compliance and kernel-level (VFS) integration are not required; system runs in user-space.
- MySQL is assumed sufficient for the metadata layer.

References

- *Amazon S3 architecture article — [Building and Operating a Pretty Big Storage System](#) (object storage with decoupled metadata index and blob storage).*
- *Apache Hadoop Distributed File System (HDFS) Architecture Guide — [HDFS Design Guide](#) (NameNode / DataNode separation).*
- *MySQL documentation — [MySQL Reference Manual](#) (reference model for journal/recovery mechanism).*
- *Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne, [Operating System Concepts](#), 10th Edition, John Wiley & Sons — File-system interface, implementation, and secondary-storage structures.*
- *EXT2/EXT3 on-disk structure documentation — [Linux Kernel EXT2 Filesystem Documentation](#).*